README Sprint S04 · Python para IA

Pregunta guía: ¿cómo automatizamos lo que hemos hecho a mano?
Fechas: 3–4 de noviembre · **Horas:** 9 h (2 h teoría · 4 h práctica · 3 h PBL)
📄 **Este README en PDF** (para imprimir; descárgalo para que funcionen los enlaces): a color · ecofriendly

> **Sigue el orden de arriba abajo y abre solo lo que pone en cada fila.** Los demás documentos del sprint son para consultar si te atascas (apartado «Si te atascas»). Si el tiempo aprieta, haz solo los ejercicios marcados «en clase»: el resto es para casa.

Qué aprenderás

1. Escribir **Python** básico (listas, diccionarios, decisiones, bucles) y **funciones con pruebas**, aprendiendo lo justo en cada misión del reto «Herramientas internas».
2. Leer y escribir **CSV y JSON** controlando los errores, y trabajar en **parejas de piloto y copiloto**.
3. Entregar una herramienta con **pruebas y Git**, y comprobar con dos *katas* individuales que cada persona sabe escribir sus funciones.

Día 1 · Martes 3/11

☐	Qué haces	Abre	Min
☐	🔔 Ceremonia · Planning, brief del cliente y test de nivel (5 preguntas, anónimo)	—	15
☐	🎯 Reto · Misión 0 · Mirar la demo del cliente + curso ★ `type()` y f-strings	Tarjetas de misión	35
☐	🎯 Reto · Misión 1 · Datos: curso ★ listas y dicts, `mision_1_datos.py`, Taller 1	Tarjetas de misión	50
☐	🎯 Reto · Misión 2 · Decidir: curso ★ `if` y `for`, `mision_2_decidir.py`	Tarjetas de misión	50
☐	🎯 Reto · Misión 3 · Funciones: curso ★ funciones, Taller 2 y `kpis.py`	Tarjetas de misión	50

En clase: 3 h 20 min.

Al acabar el día, deberías poder decir:

☐ Sé leer y modificar datos en una lista de diccionarios.
☐ Sé contar con `for` e `if` y meterlo en una función con `return`.

Día 2 · Miércoles 4/11

☐	Qué haces	Abre	Min
☐	🎯 Reto · Misión 4 · Ficheros: curso ★ excepciones y CSV, Taller 3	Tarjetas de misión	50
☐	🎯 Reto · Misión 4 · `_convertir_fila` y `leer_reservas` con sus pruebas	Tarjetas de misión	50
☐	🎯 Reto · Misión 5 · Taller 4, `exportar_json`, `main` y errores a propósito	Tarjetas de misión	25
☐	🔔 Ceremonia · README, revisión con la rúbrica, entrega en `sprint-04/` y autoevaluación	—	25

En clase: 2 h 30 min.

El reto en pocas palabras

Qué os pide el cliente. El equipo de reservas de TurisData calcula sus cifras semanales **a mano en una hoja de cálculo**. Es lento y cada persona lo hace un poco distinto. Os pide una **herramienta interna**: un pequeño programa que **lea el fichero**

de reservas, calcule tres cifras importantes (KPIs), avise con claridad si el fichero está mal y guarde los resultados en un formato que otros sistemas puedan leer.

Qué vais a entregar: un repositorio Git con `kpis.py`, `test_kpis.py`, `README.md` y un `salida.json` generado con el CSV real.

Núcleo y extra. Núcleo: las funciones obligatorias hasta que `test_kpis.py` pase y el README. Extra: las partes opcionales (POO y Git básico).

Para trabajar: reto en una página. Para consultar: ficha completa del reto (pasos, rúbrica, ayudas y errores frecuentes). Plantillas, datos y autoevaluación: `3_Reto/4_Plantillas_y_entrega/`.

✅ Antes de cerrar el sprint (Definition of Done)

Archivos subidos en el repositorio del equipo, carpeta `sprint-04/` (marca cada uno cuando lo veas en GitHub): ☐ `kpis.py` · vuestro código, con las funciones completas
☐ `test_kpis.py` · el dado, **sin cambios**
☐ `README.md` · rellenado, con la sección «Revisión con la rúbrica»
☐ `salida.json` · generado con el CSV real
☐ `autoevaluacion_nombre.md` · **uno por persona** (la coevaluación va dentro del mismo fichero): plantilla

Total en `sprint-04/`: 4 ficheros del equipo + 1 hoja por persona (un equipo de 3 sube 7). Aparte: el CSV en `data/` si aún no está, y los extras (`kpis_pandas.py`, pruebas propias) solo si los hacéis. La *review* y la *retro* no se suben.

Comprobaciones antes de dar el sprint por cerrado: ☐ `python test_kpis.py` pasa todas las pruebas
☐ `python kpis.py reservas_turisdata.csv salida.json` genera el JSON con los 3 KPIs y `calidad`
☐ Funciones con docstring y README con las definiciones de los KPIs
☐ Al menos 3 *commits* en el repositorio, de personas distintas
☐ Alguien que **no escribió esa parte** ha revisado el código y el README con la rúbrica y lo ha anotado en el README

Material extra para casa

Nada de esto es necesario para entregar el reto. Va **por orden de importancia**:

1. **Asegurar lo de las misiones.** Taller de bloques: los bloques que no hayas hecho y, si no te salió, repite la **kata** del día. Curso «Python desde cero» (abrir): los **pendientes de la ruta mínima** ★ (`idx`, tuplas, `range`, `while`, funciones que llaman funciones, comprensiones, `lambda` y clases) y los proyectos «Agenda telefónica» y «Tablas de multiplicar». Para el **nivel B** de Python (escribir funciones cortas y leer un CSV con `try/except`) hay que llegar **antes del 17/11**. Al acabar cada día, **exporta tu progreso** (botón «Exportar mi progreso») y envía una captura del contador.
2. **Repasar la teoría.** Apuntes del sprint: explican paso a paso lo que usas en las misiones e incluyen el entorno con `venv` y `pip`, y Git.
3. **Ampliar** (lo que el temario oficial incluye y no se trabaja en clase). Cuadernos de Colab `S04_01`, `S04_02`, `S04_03` y `S04_04`, empezando por los ejercicios «en clase»; y `S04_05`: clases y objetos, buenas prácticas y Git.

Si te atascas

1. Lee primero la parte «Si os atascáis» de la ficha completa del reto.
2. Mira el código explicado del cuaderno en el que estás.
3. Pregunta a tu equipo y, después, al docente. Si usas IA, cuenta en el README qué te dijo y qué comprobaste.

Todo el material, por carpetas

Las carpetas están **numeradas en el orden en que las usas**. Cada documento está en `.md`; sus PDF (color y ecofriendly) están en la subcarpeta `pdf/`.

Carpeta	Qué hay
`1_Teoria/`	Los apuntes para repasar
`2_Practica/`	Los cuadernos (Colab) y su explicación línea a línea en `codigo_explicado/`
`3_Reto/`	`1_Reto_en_una_pagina` · `2_Ficha_completa` · `3_Laminas` · `4_Plantillas_y_entrega`
`4_Referencia/`	Infografías del sprint

Cómo se trabaja un reto, en todos los sprints: método de trabajo.

Temario oficial de este sprint

Unidad	Conceptos que entran
UD 1b.1 · Puesta en marcha del entorno	Instalación de Python, entornos virtuales, pip, Jupyter, VS Code; Git básico
UD 1b.2 · Fundamentos del lenguaje	Sintaxis, tipos, operadores, input/print, condicionales, bucles
UD 1b.3 · Estructuras de datos y funciones	Listas, tuplas, diccionarios, conjuntos, comprensiones · Funciones, parámetros, lambda, args/kwargs
UD 1b.4 · Módulos, ficheros, excepciones y POO	Importación de módulos, lectura y escritura de ficheros (CSV, JSON) · Excepciones; clases y objetos (comparativa con Java)

Y ahora: Sprint 5 · Matemáticas y estadística en código (miércoles 4/11).

9 h (2 h teoría · 4 h práctica · 3 h PBL).